The VACUUM Command in Delta Lake

Detailed reference notes — storage cleanup, retention, safety checks, and best practices.

1. What VACUUM Does

VACUUM removes data files that are no longer referenced by the current (or recent) state of a Delta table's transaction log, permanently freeing up storage. It also removes any file sitting in the table directory that Delta Lake does not track at all — for example, leftover files from a failed or interrupted write.

- Scans the Delta transaction log to identify files marked as 'removed' (i.e., logically deleted through DELETE, UPDATE, MERGE, or OPTIMIZE).
- Checks how long ago each file was removed, using the transaction log timestamp — not the file's OS-level modification time.
- Permanently deletes files that were removed longer ago than the configured retention period.
- Skips any directory starting with an underscore, including _delta_log, so the transaction log itself is never touched.

2. Why It's Needed (Recap: OPTIMIZE vs VACUUM)

OPTIMIZE compacts small files into larger ones for better query performance. It rewrites data but only marks the old, small files as 'removed' in the transaction log — it does not delete them from storage. This is what enables Time Travel: old versions still physically exist. VACUUM is the command that actually deletes those old files once they are no longer needed for time travel or recovery.

Aspect	OPTIMIZE	VACUUM
What it targets	Small/fragmented data files	Stale/unreferenced data files
Action	Compacts files into larger ones	Physically deletes old files
Effect on log	Marks old files as removed (logical)	Removes files matching 'removed' entries older than retention
Effect on Time Travel	None — history preserved	Reduces/limits how far back you can time travel
Primary goal	Query performance	Storage cost reduction
Runs automatically?	Only with Predictive Optimization enabled	Only with Predictive Optimization enabled

3. Retention Period — The Core Safety Mechanism

By default, VACUUM only deletes files that have been in a 'removed' state for at least 7 days (168 hours). This threshold is controlled by the table property delta.deletedFileRetentionDuration.

```
ALTER TABLE table_name SET TBLPROPERTIES ('delta.deletedFileRetentionDuration' =
'30 days');
```

Increasing this value lets you time travel further back, at the cost of extra storage. Decreasing it reclaims storage sooner, but shortens the Time Travel window and increases risk.

Why 7 days by default?

Long-running jobs or streaming queries may reference files that are already 'logically' removed but still mid-read.
A 7-day buffer protects against deleting files that a long job still needs, which would otherwise cause read failures or data loss.
Databricks strongly recommends never going below 7 days in production.

4. The Safety Check (Danger Zone)

Delta Lake will refuse to run VACUUM with a very short or zero retention period unless you explicitly disable the built-in safety check. This prevents accidental permanent data loss from a mistyped command.

```
VACUUM my_table RETAIN 0 HOURS;-- ERROR: retention period is too short
```

To override (only when you are certain no long-running operation is touching the table):

```
SET spark.databricks.delta.retentionDurationCheck.enabled = false;VACUUM my_table
RETAIN 0 HOURS;
```

Danger Zone

Setting retention to 0 hours deletes every file not part of the very latest table version.
This permanently breaks Time Travel to any earlier version.
Only ever done in controlled scenarios — e.g., clearing sensitive/PII data for compliance (GDPR 'right to be forgotten'), or immediately after a one-off table rebuild.

5. Syntax Reference

```
VACUUM table_name { { FULL | LITE } | DRY RUN } [...]
```

Parameter	Description
table_name	Identifies the existing Delta table to vacuum. Cannot include a temporal (time travel) or options specification.
DRY RUN	Returns a list of up to 1000 files that would be deleted, without actually deleting anything.
FULL	(DBR 16.1+) Default mode. Deletes data files outside the retention window AND all untracked files in the table directory not referenced by the table.
LITE	(DBR 16.1+) Uses the Delta transaction log (not a directory listing) to find and remove files no longer referenced within the retention window. Faster, but fails with DELTA_CANNOT_VACUUM_LITE if the log has been pruned.

FULL vs LITE (Databricks Runtime 16.1+):

- FULL (default): Lists every file in the table directory and cross-checks against the transaction log. Also removes untracked/orphaned files. More thorough, but heavier on storage I/O.
- LITE: Uses only the transaction log to find files to remove — faster, cheaper, but cannot clean up untracked/orphaned files, and fails if the log has already been pruned beyond the retention window.

6. Recommended Production Workflow: DRY RUN First

Production teams never run VACUUM blind. The standard safe pattern is:

- Step 1 — Preview: Run VACUUM table_name DRY RUN to see exactly which files (up to 1000) would be deleted.

```
VACUUM sales_delta DRY RUN;
```

- Step 2 — Review: Confirm the file list looks reasonable (right table, right time range, nothing unexpected).
- Step 3 — Execute: Run the actual VACUUM once confident.

```
VACUUM sales_delta;-- or, with a custom retention:VACUUM sales_delta RETAIN 168 HOURS;
```

- Step 4 — Schedule, don't embed: Run VACUUM as a separate scheduled maintenance job, not hardcoded inside every pipeline run — this avoids redundant runs and unexpected retention overrides.

7. Automation: Predictive Optimization

On Unity Catalog managed tables, if Predictive Optimization is enabled, Databricks automatically triggers VACUUM (and OPTIMIZE) in the background on an appropriate schedule. In most modern Databricks environments, this means manual VACUUM scheduling is only needed for tables outside Unity Catalog management, or when a custom retention policy is required.

8. Key Risks & Best Practices Summary

- Never set retention below 7 days in production without a very specific, reviewed reason.
- Always run DRY RUN before the first VACUUM on a new/unfamiliar table.
- Remember: once VACUUM deletes a file, Time Travel to any version depending on that file is permanently broken — there is no undo.
- Keep VACUUM as a separate, scheduled maintenance step — not inline in transformation code.
- If long-running jobs (multi-day) write to the table, keep retention comfortably longer than the longest job duration.
- Use LITE mode for routine, fast cleanups; use FULL when you also need to catch orphaned/untracked files.

Sources

- Databricks Docs — VACUUM (SQL language reference): docs.databricks.com/aws/en/sql/language-manual/delta-vacuum
- Databricks Docs — Remove unused data files with vacuum: docs.databricks.com/aws/en/delta/vacuum
- Delta Lake Docs — Table utility commands: docs.delta.io/delta-utility